

AGENTIC ENGINEERING IN PRACTICE

Beyond the Agentic Engineer

When the whole lifecycle goes agentic

SPEAKER

Bartosz Nadworny
Staff Engineer

BEYOND THE AGENTIC ENGINEER

2026

TODAY

From one engineer's loop to an agentic lifecycle

01	Where we left off	The agentic engineer, in three slides	10'
02	The agentic lifecycle	How the practice scaled from individuals to teams and organisations	20'
03	Keeping up	Staying current in a field that changes monthly — what works, what doesn't	10'
–	Q&A	Your questions	15'

01

Where we left off

The Rise of the Agentic Engineer — the three ideas worth keeping.

THE END OF VIBE CODING

The job moved from writing code to engineering the environment that writes it

THEN

Traditional engineer

Writes code in an IDE. Craftsman mindset. Output: functions.

TRAP

Naive prompter

Chats with AI, copies snippets. Transactional. Output: AI slop and compounding rework.

NOW

Agentic engineer

Designs systems of agents, rules and checks. Output: validated workflows that produce code.

Maturity was never about prompting harder. Every step up meant **less real-time intervention** and **more upfront investment** — and determinism beat novelty every time.

The hottest new programming language is English.

— Andrej Karpathy, January 2023

VOCABULARY WE BUILT

Context is a budget. Primitives spend it wisely.

THE CONSTRAINT

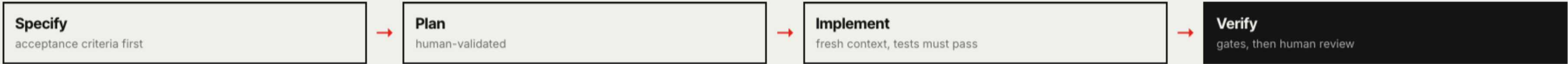
The context budget

- A window is a budget, not storage
- Quality drops sharply as it fills
- Every always-on rule is a tax on every turn

THE PRIMITIVES

Five building blocks

- **Instructions** — always-on directives
- **Skills** — know-how, loaded only when needed
- **Agents** — personas with a scoped perspective
- **Hooks** — deterministic, event-driven automation
- **MCP** — the bridge to external systems



THREE HABITS THAT HELD UP

Still true. They just scaled.

01

Encode, don't repeat

Every correction becomes a rule or a skill — not a better prompt next time.

Last time the unit of work was **one engineer in one session**. What follows is what happens when these habits are applied to **a whole delivery organisation**.

02

Invest in the environment

Hooks, tests and CI do the supervising so you don't have to watch every step.

03

Verify deterministically

The model that wrote the bug is a poor judge of it. Machine checks beat self-review.

02

The agentic lifecycle

From one engineer's loop to how a whole organisation delivers software.

WHAT ACTUALLY CHANGED

Code got cheap. Confidence didn't.

~~PRODUCE~~ → **PROVE**

Generating code is now nearly free. What's scarce is knowing it's correct. AI scales speed **and** fragility together, so the checks have to be automatic, objective and cheap to run: a **pass** or **fail**, never a feeling that it looks finished.

SAME IDEA, BIGGER SCOPE

The loop became a lifecycle

THEN • ONE ENGINEER • ONE SESSION



NOW • MANY ROLES • WEEKS OF WORK



Explicit steps, verifiable outputs, a human where it matters — the same recipe, now applied to **every station** of delivery, not just to coding.

NOTHING GETS THROWN AWAY

Keep the map. Add a layer of questions.

THE CLASSIC VIEW

Which team is responsible for this activity?

THE AGENTIC VIEW

What does the agent prepare, what proves it's right, and where does a person sign off?

Roles and responsibilities stay where they are. Only the **staffing and the checkpoints** of each activity evolve. Your existing CI/CD pipelines keep running unchanged — build, test and deploy still gate every stage.

GATED, EXPLICIT, AUDITABLE

Split the lifecycle at the contract

THE "WHAT"

Definition & design

- Framing, requirements, backlog, design
- Standardised across teams
- AI drafts — humans approve every artifact

Each step is **AI-drafted, human-decided** and closed by a gate that emits a machine-checkable signal. The next step only starts once the previous gate has passed — so every decision leaves a trace.

BUILD CONTRACT



THE "HOW"

Build & operate

- Build, verify, release, operate, learn
- Customisable per team and stack
- Autonomy grows with evidence

PRINCIPLES FOR TRUSTING AGENTS

Four principles, applied everywhere

01

Pass or fail, nothing fuzzy

Every checkpoint ends in a verdict a machine can produce — not an impression that the work seems finished.

03

Enforce outside the model

Instructions can be ignored; external controls can't. Put safeguards in tooling that runs whatever the agent decides.

02

Separate author from reviewer

Whatever drafts a change is never the one that approves, merges or ships it.

04

People keep the final say

Go-live decisions and changes to policy always need a human.

BEYOND THE DEVELOPERS

How each role's day shifts

ENGINEERS

Reviewing over typing

Agents propose changes inside the checks; you judge them and own the merge.

ANALYSTS · ARCHITECTS

Start from a draft

Framing, requirements and designs arrive as first versions for you to sharpen and approve.

QUALITY

Define what "good" means

Author the testing approach agents follow; take over when an independent check raises a genuine concern.

RELEASE · OPS

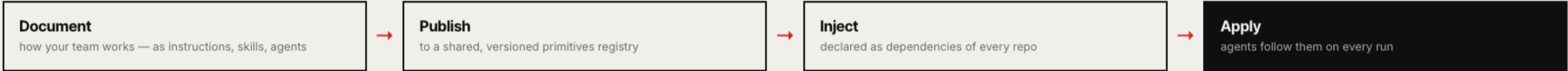
Own the decision

Agents prepare rollout plans and rehearse fixes; whether to go live remains your call.

Across every role: **put your team's working practices in writing** — that's what agents will run on.

"ENCODE, DON'T REPEAT" — AT ORGANISATION SCALE

Standards become executable



↳ learnings from every run refine the standard — and every repo picks up the new version

One team's best practice becomes **every team's default** — instead of fifty teams re-inventing the same skill in fifty slightly different ways.

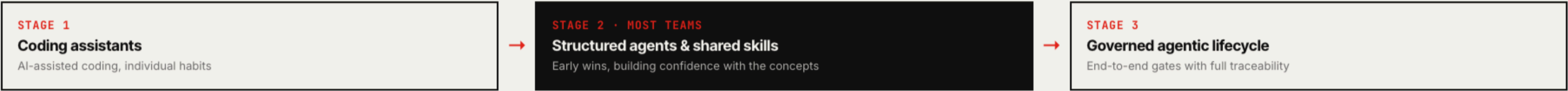
LAYERS, EACH REPLACEABLE

What sits behind the overlay

L5	Orchestration harness	Owns the lifecycle: steps, gates, roles and state. Runtime-agnostic — runs on whichever coding agent you use.
L4	Specifications	Verifiable intent before code; the source of truth agents build against (e.g. OpenSpec, Spec Kit).
L3	Execution discipline	Plan → subagents → test-first → review: how a good agent is expected to work.
L2	Primitives registry	Versioned agents, skills and instructions, shared across teams (e.g. a package manager for agent primitives).
L1	Context layer	MCP servers, live library docs and enterprise knowledge, available at runtime.

PROGRESSIVE, NOT ALL-OR-NOTHING

Teams grow into it



Going end-to-end on day one is overwhelming. Start with agents and skills to **learn the concepts and earn early wins**, then adopt the governed lifecycle where it pays off.

OPEN, NOT SOLVED

The barriers are no longer about **building** agents

01

Platform & runtime

Where do agents run securely — and how does a laptop prototype become a managed service?

02

Ownership

Anyone can build a skill in an afternoon. What's central, what's team-owned?

03

Access & cost

Repo access, licences and token spend slow adoption before anything technical does.

04

Knowledge quality

Docs are distributed and stale. How does an agent know which source is valid?

05

Evaluation

A model update can silently change behaviour. How do you catch regressions?

06

Measurement & trust

Metrics that feel like a headcount tool kill the willingness to be measured.

03

Keeping up with a moving target

Everything in section 02 will look different in six months. How do you stay current without drowning?

THE HALF-LIFE PROBLEM

Practices now expire in months, not years

MODELS

A new frontier every few months

Capabilities you designed around disappear — or appear — overnight.

Methods that ride on general capability win over hand-crafted cleverness. **Every workaround for today's model weakness is a liability on the next release.**

HARNESSES

Formats keep shifting

Rules files → instructions → skills → plugins → registries, in barely a year.

BEHAVIOUR

Silent drift

The same prompt on a new model version can behave differently — without any error.

WHAT DOESN'T WORK

Running faster on the treadmill

× CHASING EVERY LAUNCH

Tool-of-the-week churn. Nothing gets used long enough to pay off.

× BIG-BANG PLATFORM BETS

The landscape moves faster than an 18-month rollout.

× CLEVER PROMPT HACKS

They compensate for today's model and break on tomorrow's.

× MEASURING BY FEELING

In one RCT, experienced devs were ~19% **slower** with AI — while believing they were ~20% faster.

× HARD-WIRING ONE VENDOR

Your process becomes their roadmap. Switching costs a rewrite.

× LONE HEROES

Knowledge in one person's head doesn't compound — and leaves with them.

WHAT WORKS

Build on what doesn't move

✓ INVEST IN DURABLE LAYERS

Specs, tests, evals, docs, standards. Model-agnostic — and worth **more** as models improve.

✓ TIMEBOXED EXPLORATION

Protected time — even one hands-on week — plus a radar: assess · trial · adopt · hold.

✓ KEEP A GOLDEN TASK SET

A handful of your own real tasks. Re-run them on every new model or tool before switching.

✓ PRUNE AS MUCH AS YOU ADD

Delete rules and scaffolding the newer model no longer needs. Less context, better output.

✓ THIN ADAPTERS, OPEN FORMATS

MCP, AGENTS.md, skill files. Swap the engine, keep the car.

✓ LEARN AS A COMMUNITY

Fixed rhythm, named owners. Share patterns and failures, not just tool names.

IF YOU REMEMBER THREE THINGS

Beyond the agentic engineer

01

Overlay, not replacement

Keep your lifecycle and your CI/CD. Add agents that draft and gates that decide.

02

Agents draft, humans decide

Checks are automated, guardrails sit outside the model, and people make the final calls.

03

Invest in what lasts

Specs, tests, evals and context outlive any model, tool or harness.

**Probabilistic reasoning,
deterministic execution.**

Questions?

BARTOSZ NADWORNY · BARTOSZ.NADWORNY@GMAIL.COM · LINKEDIN.COM/IN/BARTOSZNADWORNY